

Part 01

1. Question: What are the benefits of using a generic sorting algorithm over a non-generic one?

* Answer:

* Type Safety: Ensures compile-time type checking, preventing `InvalidCastException` at runtime.

* Performance: Avoids boxing and unboxing overhead when dealing with value types (e.g., `int`, `double`).

* Code Reusability: Allows writing a single sorting method that works across any type implementing `Comparable<T>`.

2. Question: How do lambda expressions improve the readability and flexibility of sorting methods?

* Answer:

* Readability: Provides concise inline syntax at the call site, removing the need for separate named methods.

* Flexibility: Enables dynamic changing of the comparison logic (ascending/descending or custom fields) at runtime without modifying the underlying sort method.

3. Question: Why is it important to use a dynamic comparer function when sorting objects of various data types?

* Answer: It decouples the sorting execution logic from the evaluation criteria, allowing objects to be sorted flexibly based on different properties without modifying the underlying class structure.

4. Question: How does implementing `Comparable<T>` in derived classes enable custom sorting?

* Answer: It allows derived classes to define their own default comparison logic tailored to their specific properties while preserving type safety during comparison operations.

5. Question: What is the advantage of using built-in delegates like `Func<T, T, TResult>` in generic programming?

* Answer: It eliminates the need to declare explicit custom delegate types, reduces boilerplate code, and promotes standardization across .NET libraries and LINQ.

6. Question: How does the usage of anonymous functions differ from lambda expressions in terms of readability and efficiency?

* Answer:

* Readability: Lambda expressions are cleaner and more expressive due to type inference and => syntax compared to verbose delegate keywords.

* Efficiency: Both compile into the exact same Intermediate Language (IL) code, so there is no performance difference at runtime.

7. Question: Why is the use of generic methods beneficial when creating utility functions like Swap?

* Answer: It permits operation on any data type while avoiding performance loss from boxing/unboxing value types, eliminating duplicate overload methods.

8. Question: What are the challenges and benefits of implementing multi-criteria sorting logic in generic methods?

* Answer:

* Challenges: Managing comparison complexity, handling tie-breaker conditions, and safely checking for null objects.

* Benefits: Delivers precise, fine-grained control over complex data ordering requirements.

9. Question: Why is the default(T) keyword crucial in generic programming, and how does it handle value and reference types differently?

* Answer: It provides a type-safe default value initialization for unknown generic types: returning null for reference types and zero/false values for value types.

10. Question: How do constraints in generic programming ensure type safety and improve the reliability of generic methods?

* Answer: Constraints inform the compiler about the capabilities of T beforehand, allowing access to specific interface members and catching invalid type arguments at compile time.

11. Question: What are the benefits of using delegates for string transformations in a functional programming style?

* Answer: It supports higher-order functions, separating the iteration and execution workflow from the specific string transformation logic applied.

12. Question: How does the use of delegates promote code reusability and flexibility in implementing mathematical operations?

* Answer: It allows a single execution pipeline function to execute any arbitrary mathematical operation passed as a delegate without needing conditional branches (if/switch).

13. Question: What are the advantages of using generic delegates in transforming data structures?

* Answer: They enable transforming collections from one data type to another (T to R) with full type safety and reusability, forming the core mechanism behind mapping operations.

14. Question: How does Func simplify the creation and usage of delegates in C#?

* Answer: It removes manual delegate declarations by providing pre-defined generic delegates supporting up to 16 input parameters and a return type.

15. Question: Why is Action preferred for operations that do not return values?

* Answer: It explicitly communicates intent for void-returning side-effect operations (e.g., printing, logging) without requiring dummy return types.

16. Question: What role do predicates play in functional programming, and how do they enhance code clarity?

* Answer: Predicates encapsulate conditional evaluation returning a boolean, making filtering logic highly clear, expressive, and readable.

17. Question: How do anonymous functions improve code modularity and customization?

* Answer: They allow encapsulating localized logic right where it is needed without polluting the class with single-use named methods.

18. Question: When should you prefer anonymous functions over named methods in implementing mathematical operations?

* Answer: When the mathematical operation is simple, local to a specific context, and not intended to be reused elsewhere across the codebase.

19. Question: What makes lambda expressions an essential feature in modern C# programming?

* Answer: They form the foundational building block for declarative programming and LINQ, enabling concise, readable, and highly composable code.

20. Question: How do lambda expressions enhance the expressiveness of mathematical computations in C#?

* Answer: They express mathematical formulas inline with clean mathematical notation ($x, y \mapsto \text{expression}$), removing structural syntax noise.

Part02

2. Search about these topics:

Parallel Programming and Concurrency:

Concurrency: Managing multiple tasks simultaneously by overlapping their execution time without blocking the UI thread.

Parallel Programming: Executing multiple tasks or sub-tasks literally at the same physical time across multiple CPU cores (e.g., using `Parallel.For` or `Task Parallel Library`).

Unit Testing and Test-Driven Development (TDD):

Unit Testing: Testing individual components (functions/methods) in isolation to ensure they work as expected.

TDD: A software development methodology following a short iterative cycle:

Red: Write a failing unit test first.

Green: Write minimal production code to pass the test.

Refactor: Clean and optimize the code while maintaining passing tests.

Asynchronous Programming with `async` and `await`:

A non-blocking execution pattern used for long-running operations (like network/database I/O).

`async` modifies a method to enable asynchronous behavior.

`await` yields execution back to the caller while waiting for the task to finish, maintaining application responsiveness.